

Practical-7

Coin Change Problem Using Dynamic Programming

Theory :-

The Coin Change Problem is a Dynamic Programming problem used to find the number of different ways to make a given amount using a set of coin denominations.

In this program, each type of coin can be used any number of times. The order of coins does not matter. For example, {1, 2} to make 4 has the ways:

- 1+1+1+1
- 1+1+2
- 2+2

So, the answer is 3 ways.

Code :-

```
#include <iostream>
#include<vector>
using namespace std ;

int combCoin(vector<int>coin,int amount ){
    int m = coin.size();
    int n = amount ;
    int dp[m+1][n+1];

    // We fill the value with 1 for the amount 0
    for(int i = 0 ; i <= m ; i++){
        dp[i][0] = 1;
    }

    //if we have no coins then we can't pay means zero ways
    for (int j = 1; j <= n; j++)
    {
        dp[0][j] = 0;
    }

    // We construct whole dynamic programming table
    for (int i = 1 ; i <= m ; i++){
        for (int j = 1 ; j <= n ; j++){
            if (coin[i - 1] > j){
                dp[i][j] = dp[i - 1][j];
            }
        }
    }
}
```

```

        else{
            dp[i][j] = dp[i - 1][j] + dp[i][j - coin[i - 1]];
        }
    }
}
return dp[m][n];
}

int main (){
    int n,amount ;
    vector<int> coin;

    cout << "Enter the amount you want to pay : ";
    cin >> amount ;

    cout <<"Enter the number of varients of the coin : " ;
    cin >> n;

    coin.resize(n);

    cout << "Enter the varients of the coin : " ;
    for (int i = 0 ; i < n ; i++){
        cin >> coin[i];
    }

    int result = combCoin(coin , amount );

    cout << "Number of ways to give the amount is : " << result ;

    return 0 ;
}

```

Output :-

Output

```

Enter the amount you want to pay : 12
Enter the number of varients of the coin : 4
Enter the varients of the coin : 1
3
5
6
Number of ways to give the amount is : 14

```

Analysis Of Algorithm :- Coin Change Combination

- **Input:** Coin denominations and target amount.
- **Output:** Number of ways to make the target amount.
- **Technique:** Dynamic Programming.
- **Time Complexity:** $O(n \times M)$
- **Space Complexity:** $O(n \times M)$